ANEXO 04

PROCEDIMIENTOS SQL Y SCRIPTS

Plataforma de Gestión de Información de Privacidad (PIMS) — GESTITEL E.I.R.L.

Documentación técnica de base de datos — Documento 04

Versión del sistema documentada: migraciones core/0001 – core/0043 · customer/0001 – customer/0004

Arequipa, Perú — Octubre 2026

TABLA DE CONTENIDO

A quién va dirigido 3
Qué contiene 3
1. El sistema no tiene procedimientos almacenados, y es deliberado 3
1.1 Qué se buscó y qué se encontró 3
1.2 Por qué es una decisión de arquitectura y no una carencia 4
1.3 La contrapartida, dicha con honestidad 4
2. Dónde vive realmente la lógica: cuatro capas 5
2.1 Capa de modelo — save(): el equivalente a un disparador BEFORE INSERT/UPDATE 5
2.2 Capa de modelo — clean(): el equivalente a una restricción CHECK 7
2.3 Capa de modelo — delete(): bloqueo del borrado 8
2.4 Capa de servicios — cálculos que cruzan entidades 8
2.5 Capa de serializadores — escrituras en bloque y transacciones 10
3. La bitácora automática: el mecanismo más parecido a un disparador de auditoría 11
4. Lo que sí ejecuta el motor: restricciones declarativas 11
4.1 Las dos restricciones parciales, en detalle 12
5. Migraciones: la vía por la que cambian esquema y datos 12
5.1 Las cuatro clases de migración de datos 13
5.2 Migraciones destructivas o irreversibles — requieren respaldo previo 15
5.3 Comandos de migración 15
6. Scripts operativos 15
6.1 mocks/populate_roles.py — el único script de uso en producción 15
6.2 mocks/seeder.py y los populate_*.py — solo para demostración y pruebas 16
6.3 Comandos de administración habituales 16
7. Consultas SQL de referencia para diagnóstico 17
Resumen del documento 17
Documentos relacionados 18

A quién va dirigido

A personal técnico que necesite saber **dónde vive la lógica que transforma y valida los datos**: administradores de base de datos, equipos de mantenimiento o integración, y auditores técnicos que deban verificar que las reglas de negocio existen y son comprobables.

Qué contiene

1. **El hallazgo principal, dicho sin rodeos:** el sistema no tiene disparadores, funciones ni procedimientos almacenados. Se explica qué se buscó, cómo se verificó y por qué la ausencia es una decisión de arquitectura.
2. Dónde vive realmente la lógica, con las cuatro capas que la implementan y ejemplos de código real.
3. El inventario de servicios de cálculo.
4. Las restricciones declarativas que sí ejecuta el motor.
5. Las 47 migraciones (43 de `core` + 4 de `customer`): qué hacen, cuáles llevan lógica, cuáles son destructivas.
6. Los scripts operativos y los comandos de administración.
7. Consultas SQL de referencia para diagnóstico.

1. El sistema no tiene procedimientos almacenados, y es deliberado

1.1 Qué se buscó y qué se encontró

Se realizó una búsqueda exhaustiva sobre la totalidad del código fuente del proyecto:

Elemento buscado	Coincidencias
`CREATE TRIGGER`	**0**
`CREATE FUNCTION` / `CREATE OR REPLACE FUNCTION`	**0**
`CREATE PROCEDURE`	**0**
`RunSQL` (SQL crudo dentro de migraciones)	**0**
`.raw(...)` (consultas SQL crudas del ORM)	**0**
Archivos `.sql`	**0**
Uso de `connection.cursor()` en código de producción	**0**

> **Conclusión verificada: no existe ni un solo disparador, función o procedimiento almacenado en la base de datos, ni ninguna sentencia SQL escrita a mano en el código de producción.**

La única aparición de `connection.cursor()` en todo el proyecto está en dos archivos de pruebas automatizadas y en un utilitario de diagnóstico (`mocks/check_tables.py`), que lo usan para inspeccionar el catálogo del sistema de PostgreSQL. Ninguno forma parte del código que se ejecuta en producción.

Este documento no puede, por tanto, describir procedimientos almacenados que no existen. Lo que sí puede —y es su propósito— es documentar **dónde está esa lógica**, que existe y es abundante.

1.2 Por qué es una decisión de arquitectura y no una carencia

La lógica de negocio vive en la capa de aplicación, en Python, por cinco razones que responden a restricciones concretas de este sistema:

1. Es la única opción coherente con el modelo multiempresa. Cada empresa tiene su propio esquema PostgreSQL. Un disparador o un procedimiento almacenado habría que crearlo **N veces, una por esquema**, y volver a crearlo cada vez que se da de alta una empresa. Cualquier corrección exigiría recorrer todos los esquemas y garantizar que ninguno quedó en una versión distinta. Una función Python se escribe una vez y se aplica a todos los esquemas por construcción.

2. La lógica queda versionada junto al código que la usa. Un cambio en la fórmula del valor de un activo es una línea en un archivo del repositorio: tiene historial, autor, fecha, revisión y puede revertirse. Un procedimiento almacenado modificado en el motor no deja rastro en el repositorio, y la única forma de saber qué cambió y quién lo hizo es haber tenido la disciplina de exportarlo.

3. Es comprobable automáticamente. Todas las reglas descritas en este documento tienen pruebas automatizadas que se ejecutan en cada cambio. Probar un disparador exige una base de datos real preparada para cada caso.

4. Es portable. La aplicación depende de PostgreSQL por el modelo de esquemas y por dos características puntuales (restricciones parciales, `jsonb`), pero la lógica de negocio no está atada a ningún dialecto SQL.

5. Evita la lógica invisible. Un disparador se ejecuta sin que quien escribe la sentencia lo sepa. En este sistema, **toda transformación de datos es explícita y ocurre en un punto localizable**.

1.3 La contrapartida, dicha con honestidad

Esta decisión tiene un coste que conviene conocer, y que este documento no oculta:

> **Las reglas de negocio se aplican cuando los datos entran por la aplicación. Una escritura directa en SQL sobre la base de datos las evita todas.**

Un `UPDATE core_asset SET confidentiality = '5'` ejecutado en `psql` **no recalculará** `asset_value` ni `relevance`, porque esa fórmula vive en Python y nadie la invocó. Un `INSERT` en `core_securitylog` desde SQL **sí funcionará**, pese a que la tabla se declara inmutable, porque la inmutabilidad también vive en Python.

Lo que el motor sí garantiza **siempre**, venga la escritura de donde venga, son las restricciones declarativas del apartado 4: llaves primarias, llaves foráneas, unicidad (incluidas las parciales) y no-nulidad.

Implicación operativa: el acceso directo con privilegios de escritura a la base de datos debe considerarse una operación excepcional, restringida, registrada y realizada solo por personal que conozca estas reglas. No es un canal de trabajo.

2. Dónde vive realmente la lógica: cuatro capas

```
┌──────────────────────────┬──────────────────────────────────────────┐
│ 1. Serializadores        │ Validación de formato y escrituras en bloque │
│    (API)                 │ Transacciones que crean padre + hijos    │
├──────────────────────────┼──────────────────────────────────────────┤
│ 2. Servicios             │ Cálculos que cruzan varias entidades     │
│    core/services/        │ Única implementación de cada fórmula     │
├──────────────────────────┼──────────────────────────────────────────┤
│ 3. Modelos               │ clean() → validar antes de grabar        │
│    save() / clean()      │ save()  → calcular al grabar             │
│                          │ delete()→ bloquear el borrado            │
├──────────────────────────┼──────────────────────────────────────────┤
│ 4. Base de datos         │ Llaves, unicidad, no-nulidad.            │
│    (PostgreSQL)          │ Lo único que se cumple SIEMPRE.          │
└──────────────────────────┴──────────────────────────────────────────┘
```

2.1 Capa de modelo — save(): el equivalente a un disparador BEFORE INSERT/UPDATE

Es la capa que más se parece funcionalmente a un disparador: **se ejecuta automáticamente cada vez que se graba una fila**, sin que quien invoca el guardado tenga que acordarse.

Ejemplo mínimo — normalización. El código de un proceso siempre se guarda en mayúsculas:

```
# core/models/process_model.py
class Process(AuditModel):
    def save(self, *args, **kwargs):
        if self.code:
            self.code = self.code.upper()
        super().save(*args, **kwargs)
```

Ejemplo de cálculo con topes. La brecha y el porcentaje de cumplimiento de un requisito se derivan de los niveles de madurez actual y objetivo:

```
# core/models/gap_requirement_model.py
class GapRequirement(AuditModel):
    def save(self, *args, **kwargs):
        self.clean()

        if self.target_level is not None and self.current_level is not None:
            # La brecha nunca es negativa: superar el objetivo cuenta como cero brecha.
            self.gap = max(0, self.target_level - self.current_level)

            if self.target_level == 0:
                self.compliance = 100.00      # no hay nada que alcanzar
            else:
                # El cumplimiento nunca supera el 100 %.
                self.compliance = min(
                    100,
                    round((self.current_level / self.target_level) * 100, 2),
                )

        super().save(*args, **kwargs)
```

Ejemplo de cálculo en cadena. Al grabar un riesgo se recalculan **cinco** columnas derivadas: los dos puntajes, la reducción y las dos severidades.

```
# core/models/risk_evaluation_model.py
class RiskEvaluation(AuditModel):
    def _compute_controlled(self):
        if self.controlled_probability is None or self.controlled_impact is None:
            self.controlled_score = None
```

```
            return
        score = Decimal(self.controlled_probability) * Decimal(self.controlled_impact)
        self.controlled_score = score.quantize(Decimal('0.01'))

    def _compute_residual(self):
        ...
        self.reduction = (self.controlled_score - self.residual_score).quantize(Decimal('0.01'))

    def _compute_severity(self, matrix=None):
        from core.services.risk_service import active_matrix, resolve_severity
        if matrix is None:
            matrix = active_matrix()
        self.controlled_severity = resolve_severity(self.controlled_score, matrix)
        self.residual_severity   = resolve_severity(self.residual_score, matrix)

    def recompute(self, matrix=None):
        self._compute_controlled()
        self._compute_residual()
        self._compute_severity(matrix=matrix)

    def save(self, *args, **kwargs):
        self.recompute()          # ← siempre, antes de escribir
        super().save(*args, **kwargs)
```

Ejemplo con dependencia de configuración de empresa. El valor de un activo se calcula con una de tres fórmulas, y la relevancia solo se evalúa si el interruptor de la empresa está encendido:

```
# core/models/asset_model.py
def compute_asset_value(self):
    """S (promedio simple): (C+I+D)/3 · M (máximo): max(C,I,D)
       W (ponderado): C·wC/100 + I·wI/100 + D·wD/100"""
    ...

def compute_relevance(self, asset_value):
    from core.services.system_settings_service import fonafe_enabled
    if not fonafe_enabled():
        return ''                       # interruptor apagado → no se evalúa la fórmula
    if asset_value is None or asset_value <= RELEVANCE_THRESHOLD:
        return ''
    level = c1 * Decimal('0.3') + c2 * Decimal('0.35') + c3 * Decimal('0.35')
    return 'CRÍTICO' if level > RELEVANCE_THRESHOLD else 'NO CRÍTICO'
```

Ejemplo de generación de códigos correlativos — el caso que más se parece a una secuencia de base de datos. No se usan secuencias de PostgreSQL porque los códigos deben reiniciar por año o por registro padre, y porque deben contar también las filas eliminadas lógicamente:

```
# core/serializers/risk_evaluation_serializer.py
def _next_risk_code(self, period):
    # `all_objects` incluye las filas borradas lógicamente, porque la restricción
    # de unicidad es de base de datos y también las ve.
    last = (
        RiskEvaluation.all_objects.select_for_update()   # bloqueo hasta el fin de la transacción
        .filter(period=period)
        .aggregate(Max('risk_code'))
        .get('risk_code__max')
    )
    n = int(str(last).split('-')[-1]) if last else 0
    return f'R-{n + 1:03d}'
```

`select_for_update()` bloquea las filas leídas hasta el final de la transacción, de modo que dos usuarios creando un riesgo a la vez no pueden obtener el mismo número. Es el mecanismo que en un procedimiento almacenado sería un `SELECT ... FOR UPDATE` explícito.

Mismo patrón para `INC-###` (incidentes), `C-###` (consentimientos), `CAP-###` (capacitaciones), `CERT-###` (certificados), `AUD-AAAA-###` (auditorías, reinicia por año), `ARCO-AAAA-####` (solicitudes, reinicia por año) y `PA-###` (planes, reinicia en cada riesgo).

2.2 Capa de modelo — clean(): el equivalente a una restricción CHECK

`clean()` valida **antes** de grabar y, si algo no cuadra, aborta la operación con un mensaje en español que llega al usuario. Cubre reglas que una restricción `CHECK` de SQL no podría expresar, porque necesitan consultar otras tablas.

Reglas implementadas en `clean()`:

Tabla	Regla
`core_asset`	Los valores de confidencialidad, integridad y disponibilidad están entre 1 y 5; con el método ponderado los tres pesos suman exactamente 100.
`core_asset`	La categoría, la clasificación y la frecuencia de uso deben existir y estar activas en su catálogo configurable.
`core_riskevaluation`	La amenaza y la vulnerabilidad deben existir y estar activas en su catálogo.
`core_evaluationmatrix`	La dimensión es 3, 4 o 5; hay tantos niveles como la dimensión en cada eje; las cuatro bandas de riesgo son consecutivas.
`core_appraisalrange`	El valor mínimo no supera al máximo, y ninguna edición puede dejar un hueco nuevo sin cubrir en el rango 0,00 – 5,00.
`core_riskperiod`	La fecha de fin no es anterior a la de inicio.
`core_audit`	La fecha de fin no es anterior a la de inicio; el avance está entre 0 y 100.
`core_training`	La fecha de fin no es anterior a la de inicio; el puntaje mínimo está entre 1 y 100.
`core_arcorequest`	Si actúa un representante, sus datos mínimos son obligatorios; la respuesta no puede ser anterior a la recepción.
`core_incident`	Cada código de las listas de selección múltiple pertenece a su catálogo.
`core_document`	La categoría existe en el catálogo `DOCUMENT_CATEGORY`.
`core_risktreatmentactivity`	La fecha de cierre no es anterior a la de inicio.

El patrón de «grandfathering» — una regla no obvia

Varias de estas validaciones incorporan una excepción deliberada: **si el valor de un registro existente no cambió, se acepta aunque el ítem del catálogo se haya desactivado o borrado después**. La validación estricta solo se aplica al crear o al **cambiar** ese campo.

```
# core/models/risk_evaluation_model.py
if self.pk is not None:
    previous_value = (
        RiskEvaluation.all_objects.filter(pk=self.pk)
        .values_list(field_name, flat=True).first()
    )
    if previous_value == value:
        continue        # No cambió en un registro existente: se acepta.
```

Sin esta excepción, retirar un valor del catálogo haría **imposible editar cualquier otro campo** de todos los registros históricos que lo usaban. Con ella, el dato histórico se conserva y el valor retirado simplemente deja de ofrecerse en los formularios nuevos.

2.3 Capa de modelo — delete(): bloqueo del borrado

Cinco tablas redefinen el borrado para impedirlo. Es el equivalente a un disparador `BEFORE DELETE` que lanza una excepción:

```
# core/models/security_log_model.py
def save(self, *args, **kwargs):
    if self.pk is not None:      # tiene llave primaria → sería una modificación
        raise ValueError("SecurityLog es inmutable: no se permite modificar registros.")
    super().save(*args, **kwargs)

def delete(self, *args, **kwargs):
    raise ValueError("SecurityLog es inmutable: no se permite eliminar registros.")
```

Tabla	Comportamiento
`core_securitylog`	Solo admite inserciones. Modificar y borrar están bloqueados.
`core_consenthistory`	Ídem.
`core_incidenthistory`	Ídem.
`core_arcohistory`	Ídem.
`core_systemsettings`	El borrado está bloqueado; la llave primaria se fuerza siempre a 1, de modo que nunca puede haber una segunda fila. La edición sí está permitida.
`customer_platformaisettings`	Mismo patrón, en el esquema `public`: fila única con la configuración global del asistente de consultas teóricas.

```
# core/models/system_settings_model.py
def save(self, *args, **kwargs):
    self.pk = 1                  # patrón singleton: siempre la misma fila
    super().save(*args, **kwargs)

def delete(self, *args, **kwargs):
    raise ValueError("No se puede eliminar la configuración del sistema.")
```

2.4 Capa de servicios — cálculos que cruzan entidades

La regla de arquitectura es explícita: lo que se calcula a partir de **una sola fila** vive en el `save()` del modelo; lo que necesita **cruzar entidades** vive en un servicio, en una sola implementación compartida por todos los consumidores.

El motivo es concreto: si la pantalla de indicadores y la exportación a Excel implementaran por separado el mismo cruce, podrían dar cifras distintas para lo mismo. Por eso la pantalla, el filtro, el reporte y la simulación de impacto **llaman a la misma función**.

Inventario de servicios de cálculo

Servicio	Qué resuelve
`risk_service.py`	Matriz de evaluación activa; **traducción de un puntaje a severidad**; recálculo masivo de severidades; marca de «requiere revisión».
`risk_board_service.py`	Consulta compartida del tablero de activos de la pestaña «Evaluación de Riesgos» (misma consulta para el listado y las tarjetas).
`asset_appraisal_service.py`	Cruce del valor de un activo con las franjas de tasación (devuelve etiqueta y color).
`appraisal_coverage_service.py`	Comprueba que las franjas de tasación no dejen huecos sin cubrir.

Servicio	Qué resuelve
`risk_asset_scope_service.py`	Qué activos entran al módulo de Riesgos según el ajuste de la empresa.
`risk_treatment_scope_service.py`	Qué riesgos llegan a la bandeja de Tratamiento.
`risk_period_service.py`	Periodo activo, filtro por periodo, si un periodo está en solo lectura, copia de la metodología al cerrar.
`risk_carry_over_service.py`	Arrastre de riesgos de un periodo al siguiente: analiza primero, escribe después.
`gap_matrix_dashboard_service.py`	Agrega el cumplimiento por cláusula y del Anexo A (consumido también por el Dashboard Ejecutivo, dominio Cumplimiento).
`system_settings_service.py`	Lectura de los interruptores de empresa.
`document_limit_service.py`	Cuenta cuántos archivos tiene cargados la empresa en Gestión Documental y lo compara contra `customer_client.document_limit`.
`consent_status_service.py`	Estado efectivo de un consentimiento (Otorgado/Revocado/Expirado), derivado sin columna persistida.
`tenant_user_service.py`	Resolución de usuarios dentro del esquema de la empresa (p. ej. usuarios asignables a una capacitación, sin exponer el listado completo de `auth_user`).
`training_exam_service.py`	**Corrección del examen en el servidor** y emisión del certificado al aprobar.
`training_certificate_service.py`	Generación del PDF del certificado sobre la imagen de fondo.
`security_log_service.py`	Escritura de la bitácora (`log_action`).
`incident_notification_service.py`	Envío del correo de notificación de un incidente.
`personal_data_impact_assessment_service.py`	Crea o restaura automáticamente la EIPD de un activo en cuanto se marca como «de datos personales» (`ensure_assessment_for`).
`dashboard_service.py`	Punto de entrada único del Dashboard Ejecutivo: decide qué pestaña e indicadores son visibles para el usuario y agrega la respuesta.
`dashboard_metrics_service.py`	Calcula los 20 indicadores, **reutilizando** el servicio de cada módulo dueño — nunca recalcula una fórmula que ya existe.
`dashboard_charts_service.py`	Prepara los datos de las 9 gráficas del Dashboard Ejecutivo.
`dashboard_snapshot_service.py`	Captura perezosa de la foto mensual (una vez por mes, la primera vez que alguien abre la pantalla).
`dashboard_target_service.py`	Lectura/escritura de las metas y umbrales de aviso configurables por empresa.
`excel_service.py` y 8 servicios más	Exportación e importación de archivos Excel (motor genérico + un servicio propio por módulo con formato específico, incluido `personal_data_excel_service.py` del reporte de EIPD, solo exportación).
`auth_service.py`	Autenticación y mecanismo de usuario espejo.
`ai_chat_service.py`	Asistente de consultas teóricas (no accede a datos de la empresa). Lee su configuración —interruptor, modelo y clave— de la tabla `customer_platformaisettings` del esquema `public`, la mantiene 60 segundos en memoria y **descifra la clave del proveedor solo en el momento de la llamada**. Nunca propaga un error del proveedor.

> **Nota:** `core/services/` contiene **35** servicios (introspección directa del directorio). Fuera de ese directorio existe una utilidad aparte, `customer/utils/ai_key_crypto.py`, que es la **única** pieza del sistema que cifra y descifra un dato (la clave de acceso al proveedor del asistente). La clave maestra de ese cifrado no está en la base de datos ni en el código: se lee de una variable de entorno del servidor. Si falta o se cambia, el dato guardado deja de poder leerse y el asistente responde «no disponible» sin provocar ningún error; basta volver a introducir la clave desde el panel.

Ejemplo comentado: traducción de puntaje a severidad

Es la función más consultada del sistema y su historia ilustra por qué la lógica centralizada importa:

```
# core/services/risk_service.py
def round_score(score):
    """Redondea el puntaje al entero más cercano, con la regla ROUND_HALF_UP."""
    if score is None:
        return None
    return int(Decimal(str(score)).quantize(Decimal('1'), rounding=ROUND_HALF_UP))


def resolve_severity(score, matrix):
    if matrix is None or score is None:
        return None
    bands = sorted(matrix.risk_ranges.all(), key=lambda b: b.min_value)
    if not bands:
        return None

    value = round_score(score)

    for band in bands:
        if band.min_value <= value <= band.max_value:
            return band.level

    # Red de seguridad: por debajo de la banda más baja → la más baja;
    # por encima de la más alta → la más alta (criterio precautorio).
    for band in bands:
        if value <= band.max_value:
            return band.level
    return bands[-1].level
```

Dos detalles con consecuencias reales:

- **Se redondea antes de buscar la banda** porque los límites de las bandas son enteros y el puntaje puede ser decimal. Sin el redondeo, un puntaje de 4,67 no pertenecía a ninguna banda y la severidad quedaba vacía, lo que a su vez descontaba indicadores.
- **Se usa `ROUND_HALF_UP` y no el redondeo nativo de Python**, que es bancario: con él `round(4.5)` da 4 pero `round(5.5)` da 6. El mismo empate resolvería hacia arriba o hacia abajo según la paridad del entero vecino, algo indefendible ante un auditor.

Ejemplo: recálculo masivo con bulk_update

Al activar otra matriz de evaluación hay que reclasificar todos los riesgos. La operación tiene tres decisiones que conviene señalar:

```
# core/services/risk_service.py  (esquema del procedimiento)
def recompute_all_severities(matrix=None):
    # 1. Se leen las bandas UNA sola vez, no una por riesgo.
    # 2. Se recorre con .iterator() para no cargar todo en memoria.
    # 3. Se escribe con bulk_update en lotes, NUNCA con save():
    #    `updated_at` se actualiza sola al guardar, y save() marcaría todos los
    #    riesgos como "modificados por el usuario" por un cambio que el usuario
    #    no hizo en ellos.
    # 4. Los periodos CERRADOS quedan excluidos: no se reescribe la evaluación
    #    de un año ya cerrado.
    # 5. Deja UNA sola entrada agregada en la bitácora, no una por riesgo.
```

2.5 Capa de serializadores — escrituras en bloque y transacciones

Cuando se graba un registro con sus hijos (un riesgo con sus activos, sus controles y las actividades de cada control; una auditoría con su lista de verificación, sus hallazgos y las acciones de cada hallazgo), la escritura es **atómica**: se

borran y se recrean los hijos dentro de una transacción. Si algo falla en cualquier punto, **no se escribe nada** y la operación devuelve un error de validación.

```
@transaction.atomic
def create(self, validated_data):
    assets     = validated_data.pop('evaluation_assets', [])
    treatments = validated_data.pop('treatments', [])
    validated_data['risk_code'] = self._next_risk_code(validated_data.get('period'))
    evaluation = RiskEvaluation.objects.create(**validated_data)
    self._write_lines(evaluation, assets)
    self._write_treatments(evaluation, treatments)
    return evaluation
```

Es exactamente la garantía que daría un procedimiento almacenado transaccional, implementada en la capa de aplicación.

3. La bitácora automática: el mecanismo más parecido a un disparador de auditoría

Todo módulo de negocio registra automáticamente sus operaciones en `core_securitylog` (altas, cambios, bajas, exportaciones, importaciones), sin que cada módulo tenga que acordarse: el comportamiento está en la clase base de la que heredan todos los puntos de entrada de la API.

La función de escritura tiene un **contrato innegociable**:

```
# core/services/security_log_service.py
def log_action(action, request=None, user=None, ..., detail=None, ip_address=None):
    """Añade una fila a la bitácora de la empresa. NUNCA lanza una excepción."""
    try:
        ...
        SecurityLog.objects.create(...)
    except Exception:
        # Registrar la auditoría jamás puede romper la operación de negocio.
        logger.exception("No se pudo registrar la acción en la bitácora de seguridad.")
```

Si la bitácora falla, la operación de negocio continúa. Es la decisión opuesta a la de un disparador de base de datos, que abortaría la transacción completa. El criterio: un fallo al escribir el registro de auditoría no debe impedir que se registre un incidente de seguridad o se atienda una solicitud de un ciudadano.

Qué se registra y qué no. La columna `detail` contiene la referencia corta del objeto y los **nombres** de los campos modificados, nunca sus valores ni datos personales. Esto se apoya en una regla de diseño transversal: **los modelos que contienen datos personales devuelven solo su código como representación textual**, precisamente porque esa representación termina en la bitácora.

```
# core/models/risk_evaluation_model.py
def __str__(self):
    # Nunca nada más que el código: la bitácora vuelca esta representación.
    return self.risk_code or '(sin código)'
```

4. Lo que sí ejecuta el motor: restricciones declarativas

Estas reglas las aplica PostgreSQL **siempre**, venga la escritura de donde venga. Son la única garantía independiente de la aplicación.

Mecanismo	Cantidad	Detalle
Llaves primarias	52	Una por tabla, `bigserial` (47 en cada esquema de empresa y 5 en `public`; las 3 tablas puente tienen además la suya).
Llaves foráneas	245 constraints físicas en la instalación de referencia de dos empresas (115 por esquema de empresa + 15 en `public`; ver el documento **03**, apartado 3.0)	Garantizan que toda referencia apunte a una fila existente. Todas declaradas `DEFERRABLE INITIALLY DEFERRED`, sin cláusula `ON DELETE` (ver la advertencia siguiente).
Unicidad simple	21	Códigos de negocio y relaciones uno a uno.
Unicidad compuesta	7	Detalladas en el documento **03**, apartado 4.2.
Unicidad parcial	2	`unique_active_evaluation_matrix`, `unique_active_risk_period`.
No-nulidad	—	Según la definición de cada columna.
Índices explícitos	27	Detallados en el documento **03**, apartado 5.

> **Advertencia verificada.** Las reglas `CASCADE`, `PROTECT` y `SET_NULL` descritas en el documento **03 no están declaradas en el motor**: las aplica el ORM en Python. La comprobación sobre la instalación real confirma que las 245 restricciones de llave foránea usan el comportamiento por omisión `NO ACTION`. Consecuencia práctica: un `DELETE` manual en SQL sobre una fila con hijos **falla siempre**, en lugar de propagarse en cascada. Desde SQL directo el motor es más restrictivo que la aplicación, no menos.

4.1 Las dos restricciones parciales, en detalle

Son el mejor ejemplo de lógica que sí se delegó al motor, y por un motivo preciso: la concurrencia.

```
-- core_evaluationmatrix: una sola matriz activa por empresa
UNIQUE (status) WHERE (is_deleted = false AND status = 'A')

-- core_riskperiod: un solo periodo activo por empresa
UNIQUE (is_active) WHERE (is_active = true AND is_deleted = false)
```

Una comprobación equivalente en la aplicación («¿hay otra activa? entonces desactívala») es vulnerable: dos peticiones simultáneas pueden leer «no hay ninguna» y activar ambas. La restricción parcial hace que la segunda transacción **falle siempre**, con independencia del orden. La aplicación desactiva además la anterior; la restricción es la garantía de último recurso.

5. Migraciones: la vía por la que cambian esquema y datos

El esquema **nunca se modifica a mano**. Todo cambio es un archivo versionado en el repositorio, numerado, con dependencias explícitas y aplicable a todos los esquemas.

Aplicación	Migraciones	Ámbito
`customer`	4 (`0001` – `0004`)	Esquema `public`
`core`	**43** (`0001` – `0043`)	Todos los esquemas de empresa

De las 43 migraciones de `core`, **29 contienen al menos un paso de datos** (`RunPython`). Son el equivalente funcional de un script de migración SQL, escrito en Python y ejecutado una vez por esquema.

De las 4 de `customer`, **una** lleva un paso de datos: `customer/0004`, que además de crear la tabla `customer_platformaisettings` y de ampliar la lista de valores de `customer_platformlog.action` **siembra la fila única** de configuración del asistente —apagado y sin clave—. Es idempotente (si la fila ya existe no hace nada) y reversible (la reversa borra la fila y elimina la tabla). Se aplica **solo al esquema `public`**: ningún esquema de empresa se toca. No destruye datos.

5.1 Las cuatro clases de migración de datos

a) Siembra de configuración inicial (provisionamiento). Es la clase más numerosa y la que hace que una empresa nueva nazca utilizable. Todas son **idempotentes** (ejecutarlas dos veces no duplica nada) y **reversibles**.

```
# core/migrations/0022_gap_annex_a_assessment.py
def seed_gap_annex_a_assessments(apps, schema_editor):
    AnnexAControl        = apps.get_model('core', 'AnnexAControl')
    GapAnnexAAssessment  = apps.get_model('core', 'GapAnnexAAssessment')

    existing_control_ids = set(
        GapAnnexAAssessment.objects.values_list('annex_a_control_id', flat=True)
    )
    to_create = [
        GapAnnexAAssessment(annex_a_control_id=control.id, created_by=None)
        for control in AnnexAControl.objects.all()
        if control.id not in existing_control_ids       # ← idempotencia
    ]
    ...
```

El mecanismo es el idiomático de `django-tenants`: como **toda migración se ejecuta también al crear el esquema de una empresa nueva**, la misma migración siembra a los futuros y rellena a los existentes. **No se usan disparadores ni señales para esto.**

Migración	Siembra
`0009`	Las 3 matrices de evaluación predefinidas (3×3, 4×4, 5×5), con la 4×4 activa.
`0010`	Las 4 franjas de tasación (EXTREMO, ALTO, MEDIO, BAJO).
`0011`	Los 93 controles del Anexo A de ISO/IEC 27001:2022.
`0019`	El catálogo de categorías de documento y sus 5 ítems.
`0022`	Las 93 filas de la Declaración de Aplicabilidad.
`0023`	La fila única de ajustes + 3 catálogos de activos con 15 ítems.
`0024`	Las cabeceras de los catálogos de amenazas y vulnerabilidades.
`0028`, `0033`	Los catálogos de tipo de control, tipo de medida de seguridad y tiempo estimado.
`0036`	El «Periodo inicial» de riesgos de cada empresa.
`0037`	La lista de arranque de **57 amenazas** y **82 vulnerabilidades** (ver nota siguiente).
`0038`	Las **abreviaturas de las categorías de activo** (`HW`, `SW`, `INF`, `SRV`, `PER`, `INS`, `TAN`), con las que se forma el código de cada activo. A las categorías que la empresa hubiera creado por su cuenta les **propone** una abreviatura derivada de su nombre, que queda anotada en el informe de la actualización para revisarla.
`0039`	Las **20 metas por defecto** del Dashboard Ejecutivo (`core_dashboardtarget`), una por indicador.
`0041`	El catálogo `PERSONAL_DATA_OPERATION_TYPE` (8 ítems de arranque editables).
`0043`	Los **69 requisitos** de las cláusulas de gestión 4–10 de ISO/IEC 27001:2022 en Matriz de Brecha, con los niveles «Sin evaluar» (`NULL`, no `0`).

> **Nota sobre `core/0037`.** Siembra una lista de partida de amenazas y vulnerabilidades, **editable y no normativa**: la norma ISO/IEC 27005 retiró en 2022 los anexos que las listaban, así que no hay lista oficial vigente. Sus reglas de siembra son deliberadamente conservadoras: **nunca sobrescribe** una fila existente (si el código ya está, la deja intacta sin comparar ningún campo y sin ejecutar ningún `UPDATE`), **nunca restaura ni borra** una fila eliminada lógicamente, y coloca los ítems nuevos **detrás** de los que la empresa ya tuviera. Cada empresa puede reemplazar la lista entera importando un Excel desde Configuración del Sistema.

b) Transformación de datos existentes. Convierten datos al cambiar el modelo: pasar el proceso de un activo de texto a referencia real (`0004`), crear la versión inicial de cada documento existente (`0005`), recalcular valores con fórmulas nuevas (`0007`, `0030`), trasladar los datos de plan de un control a una actividad sin perderlos (`0034`) o **renumerar todos los activos** al adoptar el código automático (`0038`).

> **Nota sobre `core/0038`.** Reescribe el código de **todos** los activos existentes al formato `ACT-<abreviatura de la categoría>-<correlativo>` y guarda el valor anterior en `core_asset.legacy_code`, que es consultable desde el buscador del Inventario. Tres precisiones técnicas: **(1)** el renumerado se hace **en dos pasadas** —primero un código temporal y después el definitivo—, porque la columna tiene restricción de unicidad y asignar un código todavía ocupado fallaría a mitad de camino; **(2)** alcanza también a los activos **eliminados lógicamente**, que siguen ocupando su código en la restricción de unicidad; y **(3)** actualiza las filas **sin tocar la fecha de última modificación**, para no marcar como modificado por el usuario algo que hizo la actualización. **No borra ninguna fila y no altera ninguna relación**: los riesgos, controles y documentos apuntan al identificador interno del activo, no a su código. Es reversible y se puede ejecutar dos veces sin efecto adicional. Los activos que no tenían una categoría utilizable reciben el prefijo `ACT-GEN-` y se listan, con su código anterior, en el informe que imprime la actualización.
>
> **Nota sobre `core/0041` (EIPD).** Además de crear las dos tablas y sembrar su catálogo, esta migración hace un **relleno retroactivo** (clase b): crea la EIPD de todo activo marcado como «de datos personales» que no tenga ninguna fila (viva o eliminada). Es la misma lógica que se ejecuta automáticamente al guardar un activo — pero **congelada dentro del propio archivo de migración**, nunca importada del servicio vivo, para que el resultado de la migración no cambie si el servicio cambia después.

c) Corrección de valores. `0030` reclasifica las severidades de todos los riesgos con la regla de redondeo corregida. Merece mención por tres decisiones:

- La lógica de cálculo está **congelada dentro del propio archivo**, no importada del servicio vivo. Una migración debe producir el mismo resultado dentro de un año, aunque el servicio haya cambiado.
- Escribe con `bulk_update` **para no alterar `updated_at`**: el sistema no debe marcar como «modificados por el usuario» registros que el usuario no tocó.
- Deja **una sola** entrada agregada en la bitácora por empresa, no una por riesgo.

> **Nota sobre `core/0042` (reordenamiento físico de una llave primaria).** No encaja en ninguna de las cuatro clases anteriores: no siembra, no transforma un valor de negocio y no destruye nada — **reordena el `id`** de `core_annexacontrol` y de `core_gapannexaassessment` para que coincida con el orden de lectura oficial de la norma (A.5→A.6→A.7→A.8), aprovechando que sus llaves foráneas dependientes están declaradas `DEFERRABLE INITIALLY DEFERRED` (apartado 3.0). Lo hace con un **puente de dos saltos** (cada fila pasa primero por un `id` temporal fuera de rango y de ahí a su destino, dentro de la misma transacción) y **verifica, antes y después, que ningún control cambió de código ni perdió un solo vínculo** — si el recuento no coincide, no aplica ningún cambio. Es un ejemplo de que una migración puede tocar la integridad física de una tabla sin tocar un solo dato de negocio.

d) Limpieza y purga. Aquí están las migraciones que destruyen datos, y se listan explícitamente porque exigen respaldo previo. **Ninguna de las migraciones `core/0039`–`0043` ni `customer/0003`–`0004` pertenece a esta clase** — todas son aditivas o de siembra/relleno sin pérdida; la recomendación de respaldar antes de `migrate_schemas` sigue siendo la rutina general del proyecto, no una señal de que estas cinco destruyan algo.

5.2 Migraciones destructivas o irreversibles — requieren respaldo previo

Migración	Qué hace	Reversible
`0021`	Descarta las filas de la Matriz de Brecha que usaban valores de cláusula retirados.	**No** en los datos.
`0026`	Pone en «Sí» toda aplicabilidad del Anexo A que estaba sin decidir.	**No**: no hay forma de saber cuáles estaban sin decidir.
`0027`	Elimina la tabla de tratamientos de riesgo en su forma anterior. **Los tratamientos existentes se descartan.**	**No** en los datos.
`0029`	Vacía los ítems de ejemplo de los catálogos de amenazas y vulnerabilidades, conservando las cabeceras.	**Sí.**
`0031`	**Borra todos los riesgos y sus registros hijos de todos los esquemas.**	**No.**

> **`core/0031` es la migración más destructiva del sistema.** Va sola y en primer lugar dentro de su grupo, precisamente para que el paso destructivo sea evidente al revisar el registro de la migración. **Antes de aplicarla es obligatorio un respaldo verificado:**

```
pg_dump -U <usuario> -d gestitel_db -F c -f backup_pre_0031_<fecha>.dump
```

> El respaldo es la única marcha atrás.

Como criterio general, **cualquier cambio de esquema debe ir precedido de un respaldo**, sea o no reversible la migración.

5.3 Comandos de migración

```
python manage.py migrate_schemas --shared   # solo el esquema public
python manage.py migrate_schemas            # public + todos los esquemas de empresa
python manage.py makemigrations --check     # verifica que no falten migraciones por generar
python manage.py sqlmigrate core 0036       # muestra el SQL que ejecutaría una migración
```

`sqlmigrate` es la vía para **inspeccionar el SQL exacto** de cualquier migración antes de aplicarla.

6. Scripts operativos

No son procedimientos almacenados: son **scripts Python que se ejecutan desde la línea de comandos**, viven en el repositorio y se versionan con el resto del código.

6.1 mocks/populate_roles.py — el único script de uso en producción

Es el script operativo que importa. Recorre **todas las empresas** y, en cada una, crea o actualiza los roles definidos en `mocks/json/roles.json` con los permisos que les corresponden.

```
tenants = Client.objects.exclude(schema_name='public')
for tenant in tenants:
    with tenant_context(tenant):                        # ← sitúa la conexión en el esquema
        for rol_data in ROLES:
            group, _   = Group.objects.get_or_create(name=rol_data["nombre"])
            profile, _ = RoleProfile.objects.get_or_create(group=group)
            profile.description = rol_data.get("descripcion", "")
            profile.is_system   = rol_data.get("is_system", False)
            profile.save()

            permissions = PermissionsMapper.json_to_permissions(rol_data["permisos"])
            group.permissions.add(*permissions)         # ← aditivo: add, NO set
```

Dos propiedades deliberadas:

- **Es aditivo, no destructivo.** Usa `permissions.add()` y nunca `set()`: **no elimina ninguna asignación hecha manualmente** en la pantalla de Roles de una empresa. Un administrador que ajustó los permisos de un rol no pierde su trabajo al ejecutarlo.
- **Es idempotente.** Ejecutarlo varias veces produce el mismo resultado.

Cuándo hay que ejecutarlo. Cuando una versión nueva añade **permisos de Django nuevos** (típicamente, un módulo nuevo). Sin ejecutarlo, los roles existentes no reciben esos permisos y sus usuarios ven el módulo nuevo como no autorizado.

```
python mocks/populate_roles.py
```

6.2 mocks/seeder.py y los populate_*.py — solo para demostración y pruebas

`seeder.py` encadena trece scripts que rellenan una empresa con datos de ejemplo (`populate_usuarios.py`, `populate_procesos.py`, `populate_inventario.py`, `populate_riesgos.py`, `populate_incidentes.py`, `populate_arcop.py`, `populate_capacitaciones.py`, y los demás).

> **Estos scripts son para entornos de desarrollo y demostración. No deben ejecutarse contra datos reales de una empresa cliente.** La única excepción es `populate_roles.py`, descrito arriba, que sí está pensado para producción.

Existen además utilitarios de diagnóstico de solo lectura (`mocks/check_tables.py`, `check_groups.py`, `check_rangos.py`, `check_cats.py`, `check_stats.py`, `verify_mock.py`).

6.3 Comandos de administración habituales

```
python manage.py check                # verificación de consistencia del proyecto
python manage.py runserver            # servidor de desarrollo
python manage.py migrate_schemas      # aplicar migraciones (ver 5.3)
python manage.py collectstatic        # recopilar archivos estáticos para producción
python manage.py createsuperuser      # crear un superadministrador (esquema public)
python manage.py test core            # ejecutar la batería de pruebas automatizadas
python manage.py shell                # consola interactiva con el ORM
python manage.py dbshell              # consola psql sobre la base configurada
```

El alta de una empresa **no se hace por script**: se realiza desde el panel de administración del esquema `public`, mediante un formulario que crea en un solo paso la empresa, su dominio, su licencia y su usuario administrador inicial. El esquema y todas sus tablas se crean y se siembran automáticamente como parte de esa operación.

7. Consultas SQL de referencia para diagnóstico

Consultas **de solo lectura** útiles para verificar el estado de la base de datos. Para todas ellas hay que situarse antes en el esquema de la empresa que se quiere inspeccionar.

```
-- Situar la sesión en el esquema de una empresa
SET search_path TO nombre_del_esquema, public;
```

```
-- Listar las empresas registradas y su esquema
SELECT id, name, schema_name, is_active FROM public.customer_client ORDER BY name;
```

```
-- Licencia vigente de cada empresa (la de mayor fecha de fin)
SELECT DISTINCT ON (c.id) c.name, l.start_date, l.end_date, l.status
FROM public.customer_client c
JOIN public.customer_license l ON l.client_id = c.id
ORDER BY c.id, l.end_date DESC;
```

```
-- Versión de esquema de cada empresa (última migración aplicada)
SELECT max(name) FROM django_migrations WHERE app = 'core';
```

```
-- Cuántos registros hay realmente vs. cuántos ve el usuario (borrado lógico)
SELECT count(*) FILTER (WHERE NOT is_deleted) AS visibles,
       count(*) FILTER (WHERE is_deleted)     AS eliminados_logicamente,
       count(*)                               AS total_fisico
FROM core_asset;
```

```
-- Comprobar que solo hay una matriz y un periodo activos
SELECT count(*) FROM core_evaluationmatrix WHERE status = 'A'   AND NOT is_deleted;
SELECT count(*) FROM core_riskperiod       WHERE is_active      AND NOT is_deleted;
```

```
-- Códigos "ocupados" por filas eliminadas lógicamente
SELECT code, is_deleted, deleted_at FROM core_asset WHERE is_deleted ORDER BY code;
```

```
-- Últimos 50 asientos de la bitácora
SELECT created_at, username, action, module, object_type, object_id, ip_address
FROM core_securitylog ORDER BY created_at DESC LIMIT 50;
```

```
-- Verificar que las llaves foráneas no declaran ON DELETE (apartado 4)
SELECT confdeltype, count(*) FROM pg_constraint WHERE contype = 'f' GROUP BY 1;
-- Resultado esperado: una sola fila,  a (NO ACTION).
-- El total depende del número de empresas: 15 en public + 115 por cada esquema de empresa.
-- En la instalación de referencia (2 empresas):  a  |  245
```

> **Advertencia.** Estas consultas son de **lectura**. Escribir directamente en la base de datos evita todas las reglas descritas en los apartados 2 y 3: no se recalculan valores, no se valida nada, no se registra en la bitácora y las tablas inmutables dejan de serlo.

Resumen del documento

1. **No existen disparadores, funciones ni procedimientos almacenados.** Verificado con búsqueda exhaustiva sobre todo el código fuente: cero coincidencias.
2. Es una **decisión de arquitectura** motivada por el modelo multiempresa por esquemas, por mantener la lógica versionada junto al código y por poder comprobarla automáticamente.
3. La lógica **existe y es abundante**: vive en `save()`, `clean()` y `delete()` de los modelos, en 35 servicios (`core/services/`), en los serializadores transaccionales y en 29 migraciones de datos.

4. El motor garantiza **únicamente** las restricciones declarativas: llaves, unicidad (incluidas dos parciales) y no-nulidad. Las reglas de borrado `CASCADE`/`PROTECT`/ `SET_NULL` las aplica la aplicación, no PostgreSQL.
5. Existe **un único script operativo para producción**: `mocks/populate_roles.py`, aditivo e idempotente.
6. **Cinco migraciones son destructivas o irreversibles**, encabezadas por `core/0031`. Exigen respaldo previo verificado.

Documentos relacionados

Documento	Contenido
01 — Diagramas entidad-relación	Mapa visual por dominio funcional.
02 — Diccionario de datos	Las 52 tablas columna por columna.
03 — Esquema de la base de datos	Llaves, restricciones e índices.
05 — Políticas de acceso y seguridad	Quién puede ver, modificar o borrar cada dato.